

Lab overview

In this lab, I acquired practical skills with Run Command, a capability of AWS Systems Manager, to manage a fleet of three Amazon Elastic Compute Cloud (Amazon EC2) instances. Through the Run Command console, I ran the *AWS-RunShellScript* command document to seamlessly transition from using Apache HTTP Server to using the nginx web server across the entire fleet of hosts. Later, I am challenged to update only one of the three EC2 instances back to using the Apache HTTP Server, instead of using the nginx web server.

Objectives

By the end of this lab, I am able to do the following:

- Review the current infrastructure, which encompasses three EC2 instances and an Application Load Balancer.
- Test the DNS name of the Application Load Balancer to see the Apache HTTP Server response that's passed from one of the three instances in the Application Load Balancer target group.
- Launch a Run Command feature from the Systems Manager console with the *AWS-RunShellScript* command document to stop Apache HTTP Server and start the nginx web server.

Task 1: Review the existing infrastructure

In this task, I'll review the three EC2 instances that have been created for use in this lab. I'll also review the Application Load Balancer.

Task 1.1: Review the EC2 instances

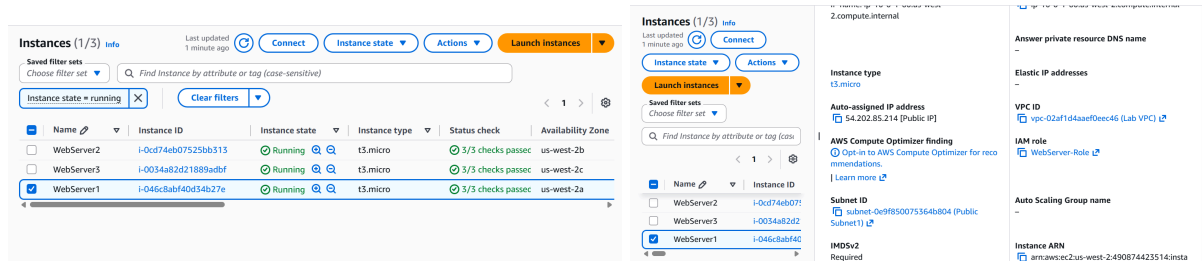
In this lab, three EC2 instances have been created by using the Amazon Linux 2 Amazon Machine Image (AMI). This AMI comes with the AWS Systems Manager Agent (SSM Agent) preinstalled. The SSM Agent is software that instances use to communicate with the Systems Manager service. I can use the SSM Agent to manage my instances through Systems Manager capabilities, such as Run Command, Inventory, State Manager, and more.

The SSM Agent makes it possible for Systems Manager to update, configure, and run commands on my instances, which can be especially helpful for batch operations or fleet management tasks. I can use it to run shell scripts, install software or patches, configure software, and gather information about operating system resources.

In addition to the SSM Agent, the instances have an instance profile attached. An instance profile is a container for an AWS Identity and Access Management (IAM) role that can be used to pass permissions to an EC2 instance at launch time. Essentially, an instance profile is a way to grant an EC2 instance permissions to perform certain actions, such as accessing Amazon Simple Storage

Service (Amazon S3) buckets, invoking AWS Lambda functions, or any other API actions that I can define in an IAM policy.

The instance profile that's attached to the three EC2 instances used in this lab has the *AmazonSSMManagedInstanceCore* policy attached to it. This policy is a managed IAM policy that grants the minimum permissions required for an instance to be managed by Systems Manager. For this lab, the policy allows me to use one of the core Systems Manager capabilities, known as Run Command operations, to run shell scripts.



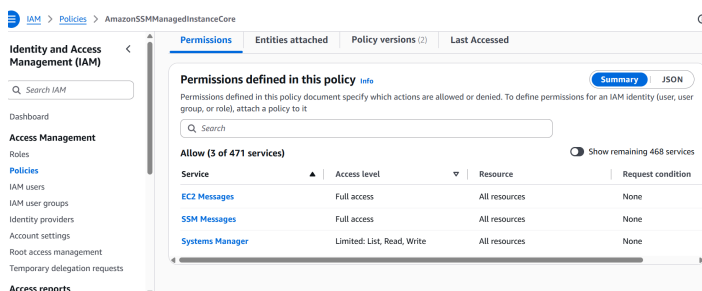
The EC2 console shows a list of three instances named *WebServer1*, *WebServer2*, and *WebServer3*. The instance states are set to *Running* and the instance types are set to *t3.micro* which is used for low-traffic web servers; development and test environments; and other small, non-resource-intensive applications.

Key settings to acknowledge for this lab include the following:

Under *Details*:

- **IAM Role:**

This is set to the *WebServer-Role*. The *AmazonSSMManagedInstanceCore* policy is attached to this role. I can verify that the policy is attached by selecting the link. Then, I can expand the *AmazonSSMManagedInstanceCore* policy to see the Systems Manager permissions that are granted by this policy.

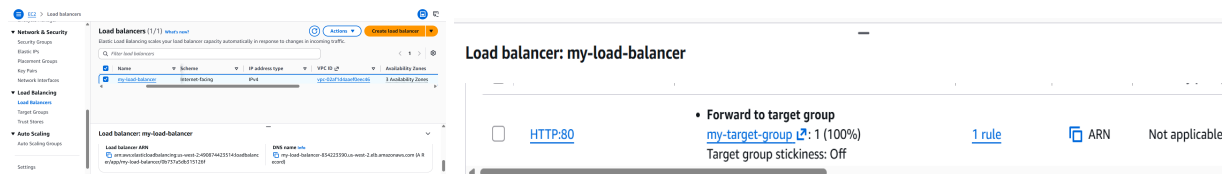


Task 1.2: Review the Application Load Balancer

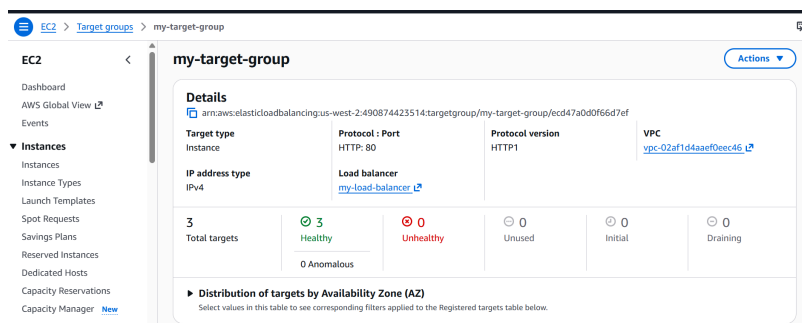
An Elastic Load Balancing (ELB) Application Load Balancer is a fully managed load-balancing service for HTTP and HTTPS traffic. It is optimized to route incoming requests to multiple targets, such as EC2 instances, in multiple Availability Zones. It operates at Layer 7 of the Open Systems Interconnection (OSI) model, which means it can make routing decisions at the application layer. Because it works at the application layer, an Application Load Balancer can support more advanced

features, such host-based or path-based routing. It can also support routing based on standard or custom HTTP headers, methods, query parameters, and source IP addresses.

For this lab, the Application Load Balancer is used to distribute incoming website traffic across multiple targets (EC2 instances) for fault tolerance and low latency. Listeners (the ports to listen on) are defined, rules (how to route requests based on content) are also defined, and then targets (such as EC2 instances) are registered so that requests can be routed to them. After the Application Load Balancer is configured, it distributes incoming client requests according to the rules that have been set. These rules help ensure that each registered target gets its share of the request load.



I noticed the load balancer has three Availability Zones. Each EC2 instance has one Availability Zone. I reviewed the **Listeners and rules** section. I can see that the **Protocol:Port** is set to **HTTP:80**. Under the **Default action**, I can see it is set to **Forward to target group**.



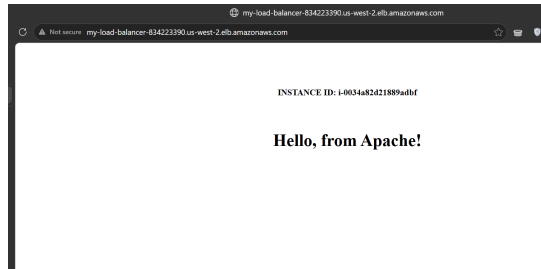
In a new browser tab, I opened the **my-target-group** link. The browser takes me to the **my-target-group** dashboard. I reviewed this dashboard and note the following details:

- **Targets:** The group has a total of three targets. The targets reference the same EC2 instances I viewed in the Amazon EC2 console.
- **Health:** All three instances should show a status of *Healthy*. If an instance fails health checks, it is deregistered from the target group and the faulty instance is removed from the list of targets that the Application Load Balancer directs traffic to.

Task complete: I have reviewed the EC2 instances used as web servers. I have also reviewed the Application Load Balancer that's used to direct web traffic to each web server.

Task 2: Test the link for the DNS name of the Application Load Balancer and update the User Data by using Run Command

In this task, I'll paste the link for the DNS name of the Application Load Balancer into a new browser tab. By opening the link in the browser, I can see the response from the targets (EC2 web servers) that are registered with the Application Load Balancer. After viewing the web servers' responses, I'll stop the Apache HTTP Server and start the nginx web server by using the Systems Manager console and Run Command to run a shell script.



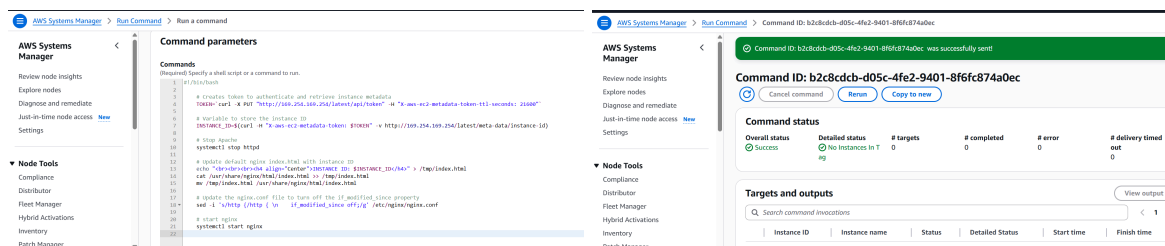
The image shows a webpage that comprises a line that reads, "Instance ID: INSTANCE_ID_VALUE", and a second line that reads, "Hello, from Apache!".

I noticed times when the ID might stay the same. However, as I kept refreshing the page, the requests filtered through the three different instance IDs. These IDs correspond to the three EC2 instances I saw in the Amazon EC2 console and in the target group for the Application Load Balancer.

This test confirms that the Application Load Balancer is redirecting web traffic bound for HTTP port 80 to the three web servers. The test also confirms that the instances are configured to run Apache HTTP Server.

Task 2.1: Use the Systems Manager Run Command feature to run a shell script

In this subtask, I'll use Run Command. I'll stop Apache HTTP Server from all three instances, and run the nginx web server on all instances with a shell script.



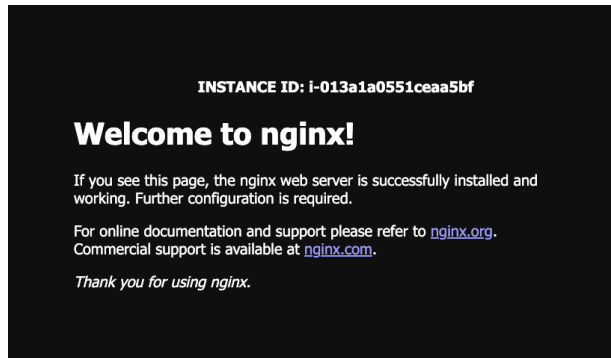
This code snippet :

- creates a token to access the instance metadata service, which is used to retrieve and store the instance ID in a variable.

- stops Apache HTTP Server, and then starts and configures the nginx web server.
- updates the existing *index.html* file to add the *instance ID* value to the top of the *index.html* file.
- updates the */etc/nginx/nginx.conf* file to turn the *if_modified_since* property to *off*.

I switched to the browser tab that's opened to the DNS name URL of the Application Load Balancer, and refreshed the page.

The browser rendered a new webpage. The top of the webpage shows *Instance ID*: followed by the ID. The webpage also includes additional lines with a *Welcome to nginx!* message, followed by the standard nginx boilerplate text, as shown below:



The image shows a webpage that comprises a first line that reads, "Instance ID: INSTANCE_ID_VALUE", and a second line that reads, "Welcome to nginx!" These two lines are followed by standard nginx boilerplate text.

When refreshing the browser, I saw that it filters through different instance IDs as the HTTP traffic is routed to different instances.

Task complete: I have successfully tested the DNS name URL of the Application Load Balancer to view the Apache HTTP Server responses to web traffic. I have updated the EC2 instance User Data to stop the Apache HTTP Server, and start the nginx web server. The User Data also updates the existing default *index.html* file to display an instance ID at the top of the page. This instance ID belongs to the instance that the HTTP traffic was sent to.

Conclusion

I now have successfully done the following:

- Reviewed the current infrastructure, which encompasses three EC2 instances and an Application Load Balancer.
- Tested the DNS name of the Application Load Balancer to see the Apache HTTP Server response that's passed from one of the three instances in the Application Load Balancer target group.
- Launched a Run Command feature from the Systems Manager console with the *AWS-RunShellScript* command document to stop Apache HTTP Server and start the nginx web server.

